Once you find a suitable value for how much memory your application needs, I recommend you use it both as a request (`resources.requests.memory`) and as a limit (`resources.limits.memory`). The reason for that is deeply connected to how the JVM works, and particularly how the JVM heap memory behaves. Growing and shrinking the container memory dynamically will affect the application's performance, since the heap memory is dynamically allocated based on the memory available to the container. Using the same value for the request and the limit ensures that a fixed amount of memory is always guaranteed, resulting in better JVM performance. Furthermore, it allows the Java Memory Calculator provided by the Paketo Buildpacks to configure the JVM memory in the most efficient way.

I've mentioned the Java Memory Calculator a few times now. The following section will expand on the subject.

CONFIGURING RESOURCES FOR THE JVM

The Paketo Buildpacks used by the Spring Boot plugin for Gradle/Maven provide a Java Memory Calculator component when building container images for Java applications. This component implements an algorithm that has been refined and improved over the years, thanks to the Pivotal (now VMware Tanzu) experience with running containerized Java workloads in the cloud.

In a production scenario, the default configuration is a good starting point for most applications. However, it can be too resource-demanding for local development or demos. One way to make the JVM consume fewer resources is to lower the default 250 JVM thread count for imperative applications. For that reason, we've been using the `BPL_JVM_THREAD_COUNT` environment variable to configure a low number of threads for the two